PROJECT REPORT

SCRIPT-CONTROLLED ACL - RESTRICT RECORD ACCESS BASED ON FIELD VALUE

1. INTRODUCTION

1.1 Project Overview

The Script-Controlled Access Control List (ACL) project is designed to restrict access to records dynamically based on the value of a selected field. Instead of applying the same access rule to every record, the system evaluates field values and uses a script to decide whether the current user is allowed to view, modify, or access a particular record. This approach provides flexible and condition-based record security for applications that contain sensitive or role-dependent information.

1.2 Purpose

The purpose of this project is to improve record-level security by introducing a script-based ACL mechanism. The solution checks a field value before granting access and prevents unauthorized users from accessing records that do not satisfy the defined condition. It aims to provide controlled, reliable, and dynamic authorization while reducing the need for multiple static access rules.

2. IDEATION PHASE

2.1 Problem Statement

Traditional access-control rules may grant or deny access mainly according to roles or general permissions, but some applications require access to depend on the actual value stored in a record. For example, records can be restricted according to department, status, ownership, classification, or approval state. Without a field-based condition, users may receive access to records that they should not be able to view or modify. The proposed project addresses this problem by using a scripted ACL condition to evaluate the relevant field value before allowing access.

2.2 Empathy Map Canvas

The main users of the system include administrators, application developers, employees, and record owners. Users need secure access to the records required for their work, while administrators need a simple way to enforce field-based restrictions. Developers need an ACL mechanism that can be customized through scripting. The solution therefore focuses on security, usability, controlled access, and clear authorization behavior.

2.3 Brainstorming

The brainstorming phase identifies possible ways to implement dynamic record protection. Ideas include checking user roles, comparing record ownership, validating department or category fields, checking record status, and combining user information with record values. Among these ideas, a script-controlled ACL provides a flexible method because the access decision can be changed according to business conditions without creating a separate rule for every record.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey map

The user journey begins when a user attempts to access a protected record. The system identifies the user, retrieves the requested record, evaluates the configured field value, executes the ACL script, and returns an access decision. If the field value satisfies the defined condition, access is granted; otherwise, access is denied. This journey ensures that authorization is performed before protected information is exposed.

3.2 Solution Requirement

The solution requires a record source containing the field used for authorization, a user identity or role context, an ACL rule, and a script that evaluates the field value. The system should support allow and deny outcomes, handle invalid or missing values safely, and ensure that unauthorized records remain inaccessible. The implementation should also be easy for an administrator or developer to maintain and test.

3.3 Data Flow Diagram

The data flow starts with a user request for a record. The request is passed to the access-control layer, where the user context and requested record are evaluated. The ACL script reads the configured field value and applies the access condition. The result is then returned as an authorization decision. An approved request proceeds to the record, while a rejected request is stopped by the security layer.

3.4 Technology Stack

The project can be implemented using a platform that supports ACL configuration and server-side scripting, such as ServiceNow. JavaScript can be used for the ACL condition or authorization logic, while the platform database stores the protected records and their field values. Browser-based testing and platform developer tools can be used to configure, execute, and verify the access rules.

4. PROJECT DESIGN

4.1 Problem Solution Fit

The proposed solution directly addresses the need for record-level access based on business data. Instead of relying only on broad user permissions, the ACL evaluates a field within the requested record. This creates a closer relationship between the security rule and the actual business requirement, helping ensure that users receive access only when the record satisfies the defined condition.

4.2 Proposed Solution

The proposed system uses a script-controlled ACL that checks a selected field value before granting access. For example, an ACL can allow access only when a record has an approved status or when its department field matches the user's department. The script returns an authorization result, allowing compliant records and denying records that fail the condition. The rule can be adapted for different field-based security requirements.

4.3 Solution Architecture

The architecture consists of the user interface, application or record layer, ACL security layer, scripting logic, and database. A user sends a record-access request through the application. The ACL layer intercepts the request and invokes the script. The script evaluates the user context and record field value and produces an allow or deny result. Only authorized requests continue to the record data.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The project can be completed through sequential activities: requirement identification, problem analysis, ACL design, scripting and configuration, test-data preparation, functional testing, performance checking, result documentation, and final report preparation. Each stage should be reviewed before moving to the next stage so that the access condition is correctly implemented and unauthorized access is prevented.

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Performance testing verifies that the scripted ACL evaluates access requests without causing unnecessary delay. Test cases should include permitted field values, restricted field values, missing or invalid values, different user roles, and repeated access requests. The expected result is that valid requests are authorized correctly while restricted requests are denied consistently, with acceptable response time.

7. RESULTS

7.1 Output Screenshots

The results section should contain screenshots showing the configured ACL, the script or condition used for field-based authorization, a record that satisfies the access condition, and a record for which access is denied. The screenshots should demonstrate that the access decision changes according to the field value and that the intended security restriction is working.

8. ADVANTAGES & DISADVANTAGES

Advantages

The approach provides dynamic record-level security, supports business-specific access conditions, reduces dependence on multiple static rules, and can be adapted to different fields and authorization requirements. It also provides a centralized method for enforcing sensitive record restrictions.

Disadvantages

Script-based ACLs require careful development and testing because an incorrect condition can unintentionally grant or deny access. Complex scripts may also be harder to maintain, and excessive scripting can affect performance if authorization checks are poorly designed.

9. CONCLUSION

Conclusion

The Script-Controlled ACL project demonstrates how record access can be restricted according to field values rather than relying only on general permissions. By evaluating the requested record and applying a scripted authorization condition, the system provides a flexible method for protecting sensitive records. Proper testing and maintenance of the ACL logic are important to ensure consistent and secure access control.

10. FUTURE SCOPE

Future Scope

The project can be extended to support multiple field conditions, user attributes, department-based policies, record ownership, approval status, and more advanced combinations of authorization rules. Future improvements may also include centralized policy management, audit logging for denied requests, reusable authorization functions, and automated security testing for ACL configurations.

11. APPENDIX

Source Code (If any)

Include the final ACL script, configuration details, test scripts, and any supporting code used to implement the field-value-based access restriction.

Dataset Link

Include the link or reference to the sample dataset used for testing the ACL conditions. If no external dataset is used, state that test records were created within the application environment.

GitHub & Project Demo Link

Include the GitHub repository and project demonstration link, if available. These links should provide access to the project implementation and demonstrate the working behavior of the script-controlled ACL.